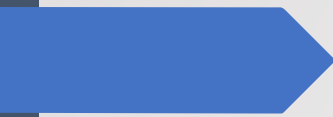




BSM422 Bilgisayarla Görme ve Görüntüleme Teknikleri

Dr. Öğr. Üyesi Mehmet Zahid YILDIRIM



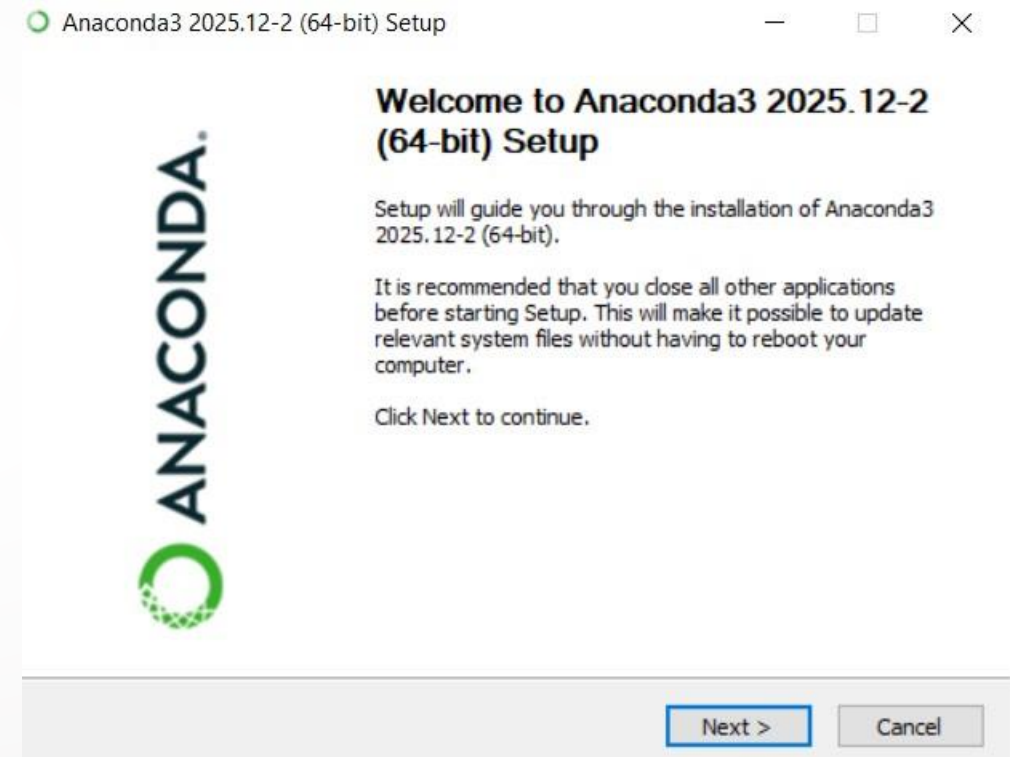
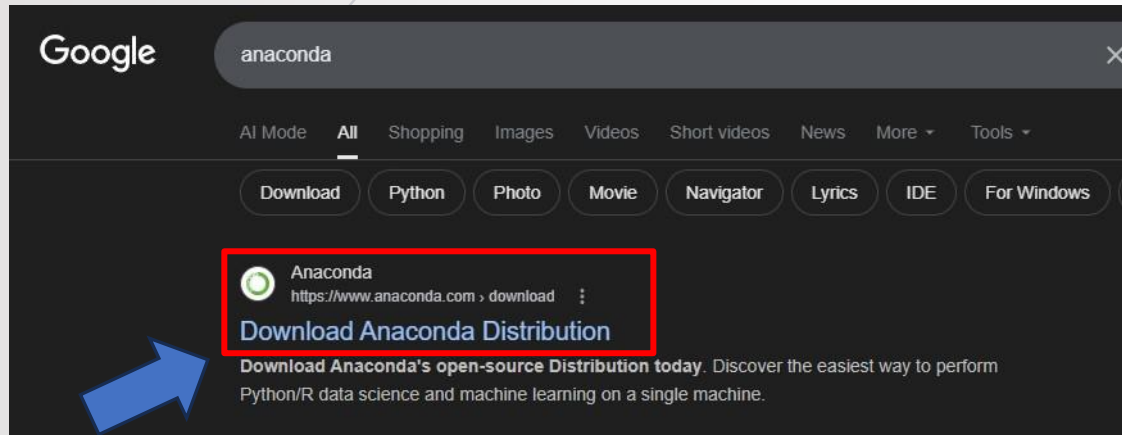
Bu ders;

Python Programlama Dili;

- Kurulumlar
- Jupyter Notebook
- Python Söz Dizimi
 - Veri Tipleri
 - Koşullu İfadeler
 - Döngüler
 - Fonksiyonlar
- Python Kütüphaneleri
 - Numpy, Pandas, Matplotlib, OS

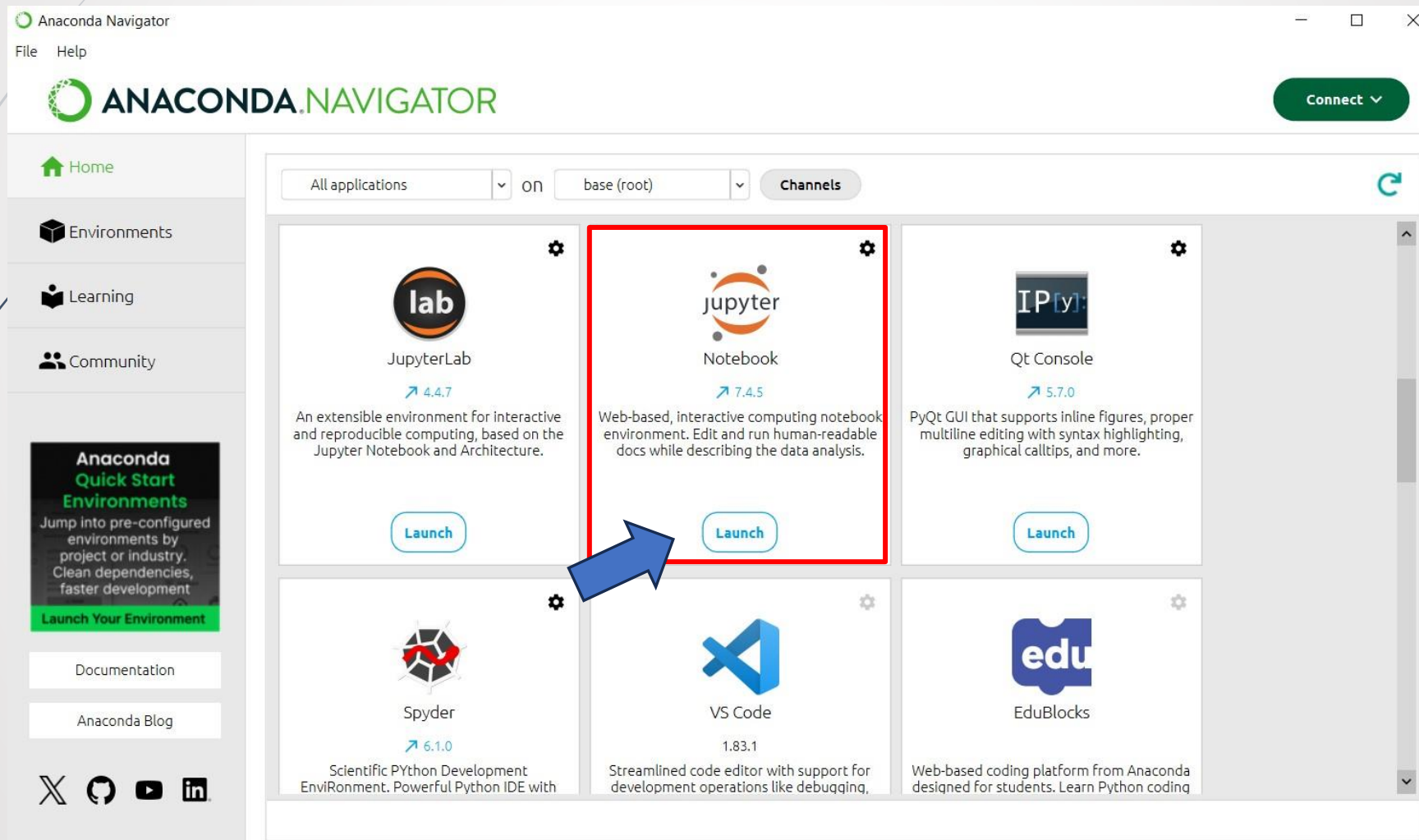
Python

Anaconda Kurulumu



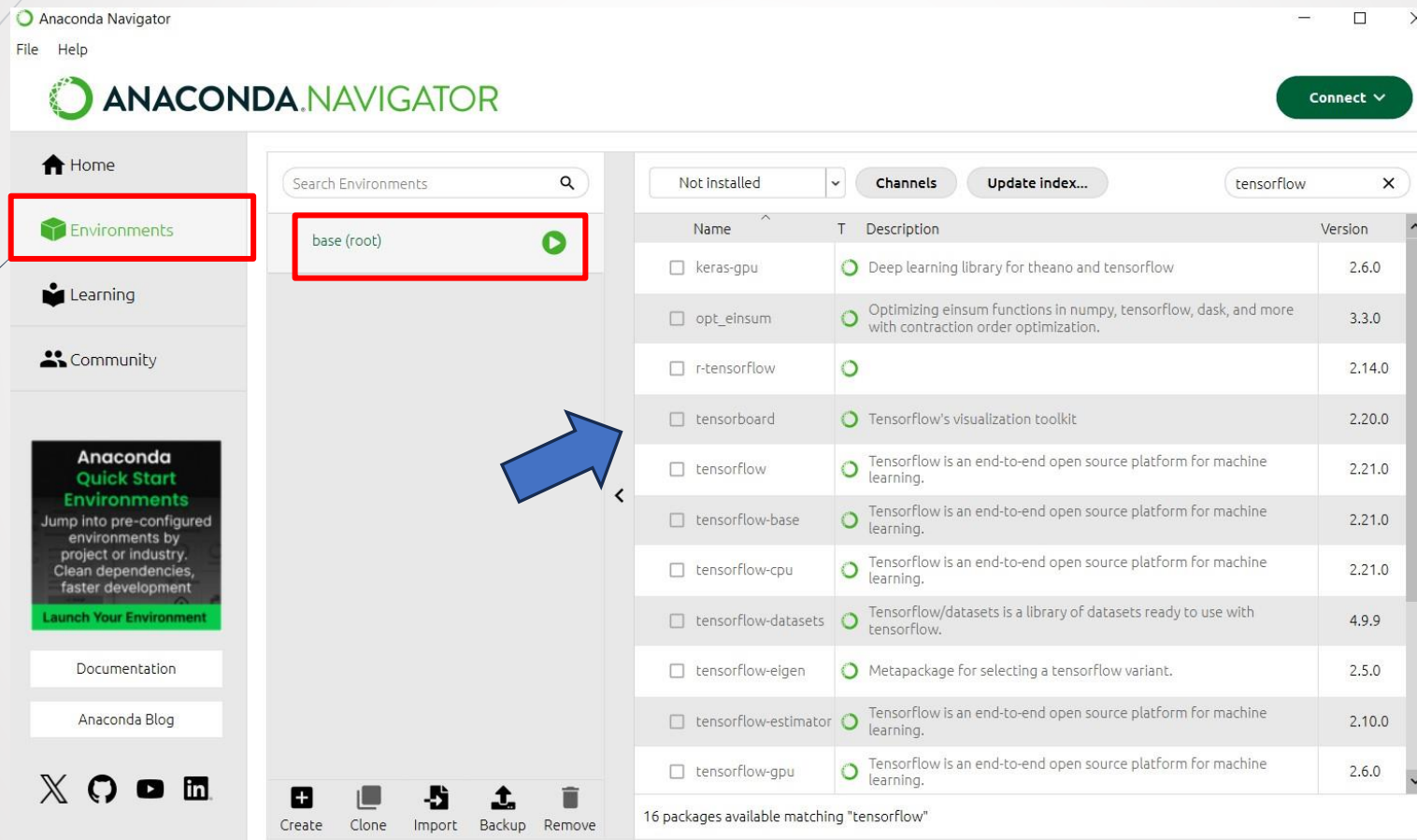
Python

Anaconda Navigator



Python

Anaconda Navigator - Environments



The screenshot shows the Anaconda Navigator application window. The left sidebar contains navigation options: Home, Environments (highlighted with a red box), Learning, and Community. Below these is a section for 'Anaconda Quick Start Environments' with a 'Launch Your Environment' button. The main panel displays the 'Environments' tab, showing a search bar and a list of environments. The 'base (root)' environment is highlighted with a red box and a green play button. A blue arrow points to the 'tensorflow' package in the list of installed packages.

Environments

Search Environments

base (root)

Channels **Update index...** tensorflow

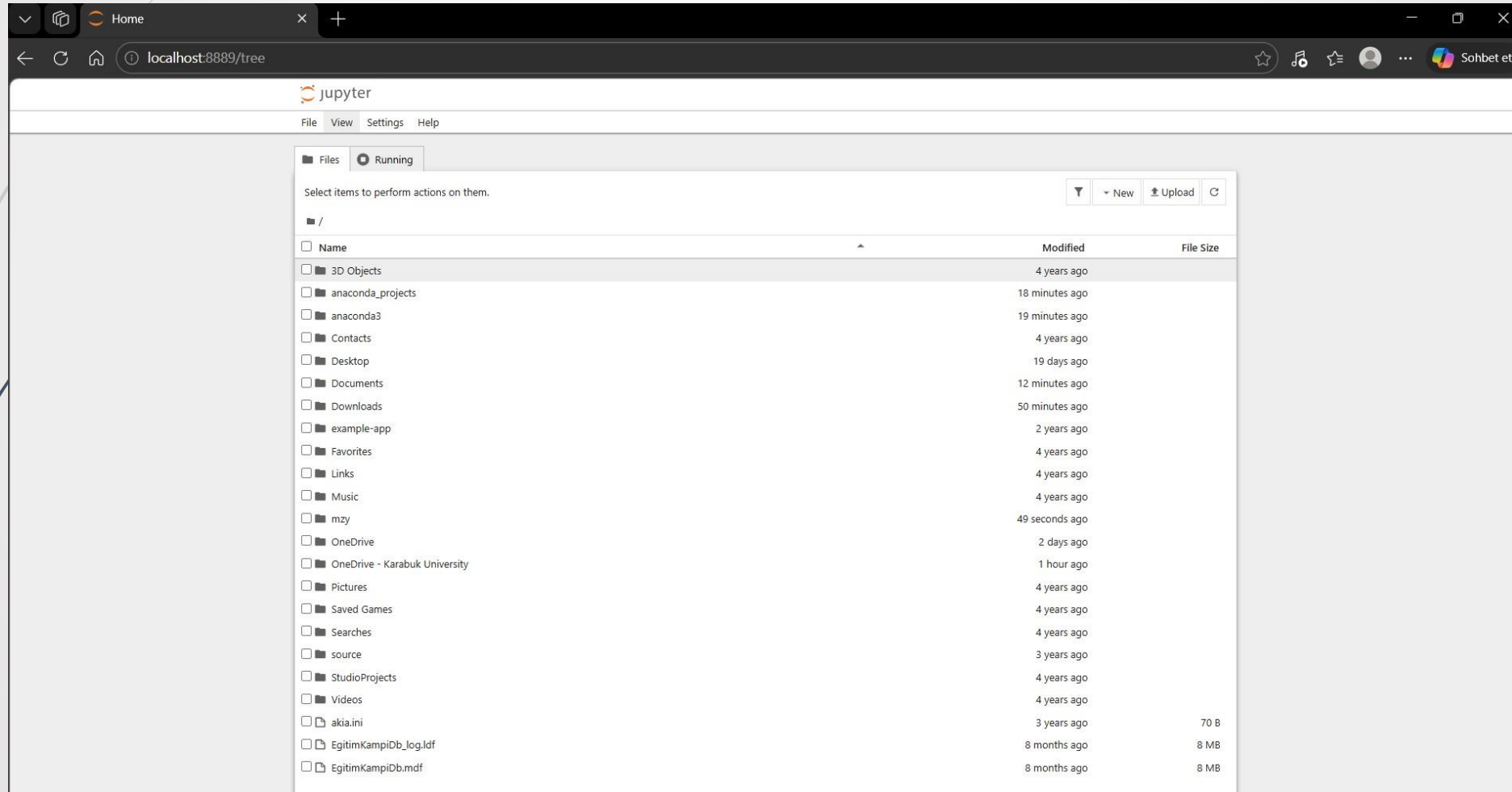
Name	Description	Version
keras-gpu	Deep learning library for theano and tensorflow	2.6.0
opt_einsum	Optimizing einsum functions in numpy, tensorflow, dask, and more with contraction order optimization.	3.3.0
r-tensorflow		2.14.0
tensorboard	Tensorflow's visualization toolkit	2.20.0
tensorflow	Tensorflow is an end-to-end open source platform for machine learning.	2.21.0
tensorflow-base	Tensorflow is an end-to-end open source platform for machine learning.	2.21.0
tensorflow-cpu	Tensorflow is an end-to-end open source platform for machine learning.	2.21.0
tensorflow-datasets	Tensorflow/datasets is a library of datasets ready to use with tensorflow.	4.9.9
tensorflow-eigen	Metapackage for selecting a tensorflow variant.	2.5.0
tensorflow-estimator	Tensorflow is an end-to-end open source platform for machine learning.	2.10.0
tensorflow-gpu	Tensorflow is an end-to-end open source platform for machine learning.	2.6.0

16 packages available matching "tensorflow"

Create Clone Import Backup Remove

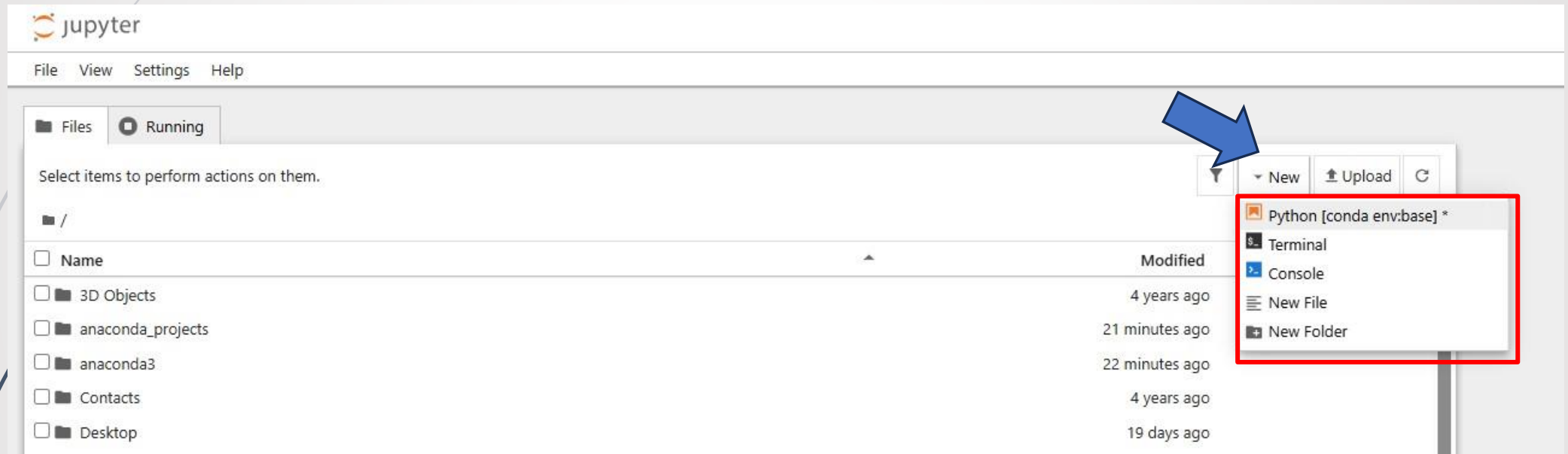
Python

Jupyter Notebook



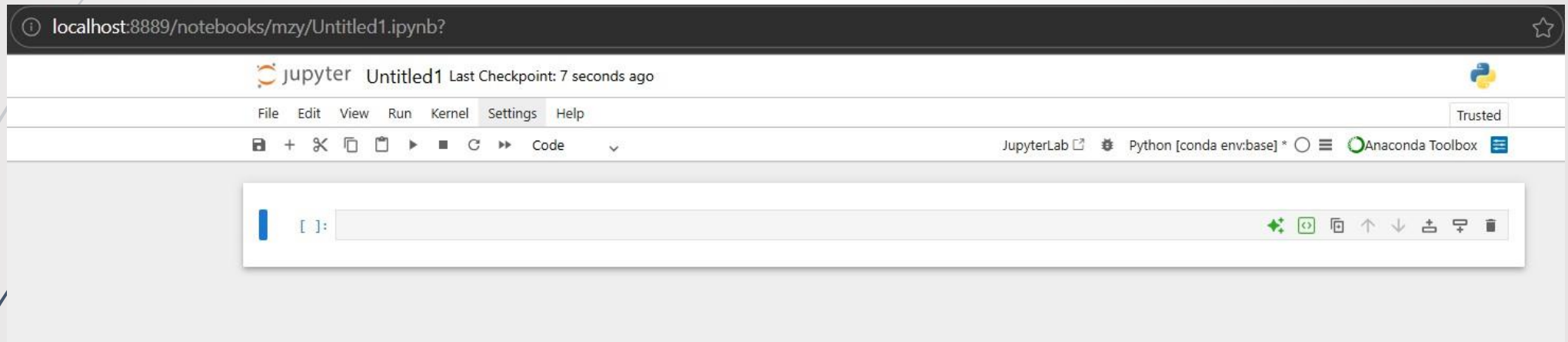
Python

Jupyter Notebook – Yeni Python Projesi



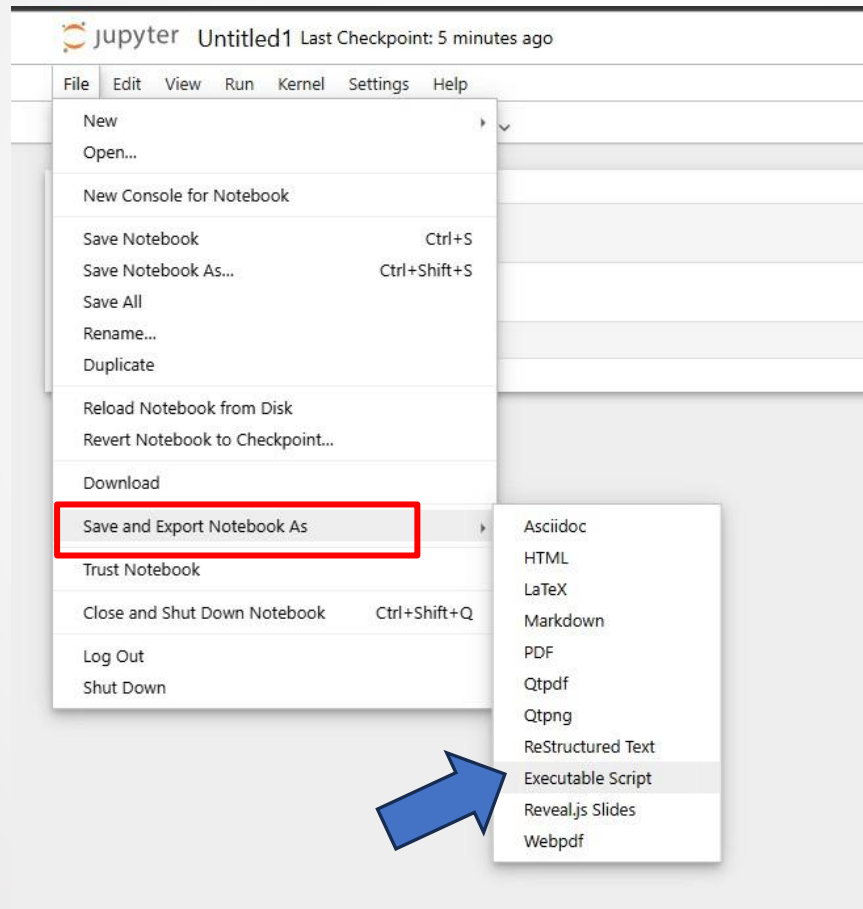
Python

Jupyter Notebook – Kod Ekranı



Python

Jupyter Notebook – Export (.py)



Python

Açıklama Satırları

```
[1]: print("hello world")
```

hello world

```
# Açıklama Satırı
```

```
"""
```

```
Karabük Üniversitesi  
Bilgisayar Mühendisliği  
Bilgisayarla Görü Dersi
```

```
"""
```

Python

Değişkenler

```
[2]: # Değişkenler
tam_sayi = 42
ondalikli_sayi = 55.78
string = "merhaba"

print(tam_sayi)

print(ondalikli_sayi)

print(string)

print(tam_sayi, " - ", ondalikli_sayi, " - ", string)
```

42
55.78
merhaba
42 - 55.78 - merhaba

Python

Dört İşlem

```
[3]: # Dört İşlem
a = 15.2
b = 7

toplam = a + b
fark = a - b
carpim = a * b
bolum = a / b

print("TOPLAM: ", toplam)

print("TOPLAM: {} FARK: {}".format(toplam, fark))

print("CARPIM: %.1f BOLUM: %.2f" % (carpim, bolum))
```

TOPLAM: 22.2
TOPLAM: 22.2 FARK: 8.2
CARPIM: 106.4 BOLUM: 2.17

Python

Değişkenler Arası Dönüşümler

```
[4]: # Değişkenler Arası Dönüşümler
i = 42
f = 34.78

print(i, f)

float_i = float(i)
int_f = int(f)

print(float_i, int_f)
```

42 34.78
42.0 34

Python

Listeler

```
[5]: # Listeler

sayilar = [10, 20, 30, 40, 50]
print(sayilar)

karisik_liste = [25, "Python", 3.14, True]
print(karisik_liste)
print("-----")

meyveler = ["elma", "armut", "muz", "çilek", "portakal"]
print(len(meyveler)) # eleman sayısı
print(meyveler[0])   # ilk eleman
print(meyveler[-1])  # son eleman
print(meyveler[:3])  # ilk üç eleman
print(meyveler[1:4]) # 2 ve 4. eleman arası
print("-----")
```

```
[10, 20, 30, 40, 50]
[25, 'Python', 3.14, True]
-----
5
elma
portakal
['elma', 'armut', 'muz']
['armut', 'muz', 'çilek']
-----
```

Python

Listeler

```
liste = [1, 2, 3]
liste.append(4)      # sona ekle
print(liste)
liste.insert(1, 10)  # indexe ekle
print(liste)
print("-----")

sayilar = [10, 20, 30, 40]
sayilar.remove(20)   # değere göre sil
print(sayilar)
sayilar.pop()        # sondan sil
print(sayilar)
del sayilar[0]        # indexe göre sil
print(sayilar)
print("-----")

sayilar = [15, 7, 21, 32, 4, 5, 18]
sayilar.reverse()
print(sayilar)
sayilar.sort()
print(sayilar)
```



```
[1, 2, 3, 4]
[1, 10, 2, 3, 4]
-----
[10, 30, 40]
[10, 30]
[30]
-----
[18, 5, 4, 32, 21, 7, 15]
[4, 5, 7, 15, 18, 21, 32]
```


Python

Tuple

```
[6]: #Tuple
# Değiştirilemez (salt okunur) veri tipidir.

tuple_ = (1, 2, 3, 4, 4, 5)
print(tuple_[0])      #ilk eleman
print(tuple_[2:])      #2.indexten sonraki elemanlar
print(tuple_.count(4)) #eleman sayısı
print("-----")
tuple_rgb = (120, 35, 79)
r, g, b = tuple_rgb
print(r, g, b)
```

1
(3, 4, 4, 5)
2

120 35 79

Python

Deque

```
[7]: #Deque
from collections import deque
dq = deque(maxlen=3)
dq.append(1)
dq.append(2)
dq.appendleft(3)
print(dq)
print("-----")
dq.clear()
print(dq)
```

→

```
deque([3, 1, 2], maxlen=3)
-----
deque([], maxlen=3)
```

Python

Dictionary

```
[8]: #Dictionary
      #Key ve Value çiftlerinden oluşan veri tipidir.

      dictionary = {"Karabük": 78,
                    "Konya" : 42,
                    "Samsun" : 55}
      print(dictionary)

      print(dictionary.keys())

      print(dictionary.values())

      print(dictionary["Karabük"])
```

→

```
{'Karabük': 78, 'Konya': 42, 'Samsun': 55}
dict_keys(['Karabük', 'Konya', 'Samsun'])
dict_values([78, 42, 55])
78
```

Python

Koşullu İfadeler

```
[9]: #Koşullu İfadeler (if-else)
sayi1 = 42.0
sayi2 = 78.0

if sayi1 < sayi2:
    print("sayi1 küçüktür sayi2")
elif sayi1 > sayi2:
    print("sayi1 büyüktür sayi2")
else:
    print("sayi1 eşittir sayi2")

print("-----")

liste = [1, 2, 3, 4, 5]
deger = 32
if deger in liste:
    print("{} sayısı listenin içerisindedir.".format(deger))
else:
    print("{} sayısı listenin içerisinde değildir.".format(deger))

print("-----")
```

sayi1 küçüktür sayi2

32 sayısı listenin içerisinde değildir.

Python

Koşullu İfadeler

```
dictionary = {"Karabük": 78, "Konya" : 42, "Samsun" : 55}
keys = dictionary.keys()
deger = "Karabük"
if deger in keys:
    print("evet")
else:
    print("hayır")

print("-----")

bool1 = True
bool2 = False

if bool1 and bool2:
    print("and doğru")

if bool1 or bool2:
    print("or doğru")
```

evet

or doğru

Python

Döngüler

```
[10]: #Döngüler (for - while)
meyveler = ["elma", "muz", "çilek"]

for meyve in meyveler:
    print(meyve)

print("-----")

for i in range(5):
    print(i)

print("-----")

for i in range(2, 10, 2):
    print(i)
```

elma
muz
çilek

0
1
2
3
4

2
4
6
8

Python

Döngüler

```
print("-----")

toplam = 0
for i in range(1, 6):
    toplam += i

print(toplam)

print("-----")

i = 0
while i < 5:
    print(i)
    i += 1
```



15

0
1
2
3
4

Python

Fonksiyonlar

```
[11]: #Fonksiyonlar
def yazdir():
    print("Merhaba!")

yazdir()

print("-----")

def yazdir(str):
    print(str)

yazdir("Karabük")

print("-----")

def topla(a, b):
    print(a + b)

topla(3, 5)
```

Merhaba!

Karabük

8

Python

Fonksiyonlar

```
def toplama(a, b):  
    return a + b  
  
sonuc = toplama(4, 6)  
print(sonuc)  
  
print("-----")  
  
def daire_alani(r, pi = 3.14):  
    return pi * r * r  
  
sonuc = daire_alani(4)  
sonuc2 = daire_alani(4, 3)  
print(sonuc, sonuc2)
```

10

50.24 48

Python

Fonksiyonlar

```
def islem(a, b):  
    toplam = a + b  
    carpim = a * b  
    return toplam, carpim  
  
t, c = islem(3, 4)  
print(t, c)  
  
print("-----")  
  
def ortalama(liste):  
    return sum(liste) / len(liste)  
  
notlar = [70, 80, 90]  
print(ortalama(notlar))  
  
print("-----")  
#-----Lambda Fonksiyon-----  
  
kare = lambda x: x**2  
print(kare(5))
```

7 12

80.0

25

Python

Yield

```
[12]: #Yield
      """
      iterasyon: Bir veri yapısının elemanlarını tek tek dolaşma işlemine denir.
      generator: Elemanları hafızada tutmadan, ihtiyaç oldukça üreten özel iterator'dür.
      yield     : Fonksiyonu generator'e dönüştürür.
      """

      liste = [1, 2, 3]
      for i in liste:
          print(i)
```

1
2
3

Python

Yield

```
generator = (x for x in range(1, 4))
for i in generator:
    print(i)

print("-----")

def createGenerator():
    liste = range(1,4)
    for i in liste:
        yield i

generator = createGenerator()
for i in generator:
    print(i)
```

1
2
3

1
2
3

Python - Kütüphaneler

Numpy

```
[13]: #Numpy (Matris İşlemleri)
      """
      - NumPy, Python'da sayısal hesaplama, matris işlemleri ve bilimsel programlama için kullanılan en temel kütüphanedir.
      - NumPy → Numerical Python.
      - Listelere göre daha hızlıdır, daha az bellek kullanır.
      """

      import numpy as np
      # Array oluşturma
      a = np.array([1, 2, 3, 4])
      print(a)
      print("-----")

      # Matris oluşturma
      matris = np.array([
          [1, 2, 3],
          [4, 5, 6]
      ])
      print(matris)
      print("-----")
```

→

```
[1 2 3 4]
-----
[[1 2 3]
 [4 5 6]]
-----
```

Python - Kütüphaneler

Numpy

```
print(matris.shape)    # satır x sütun bilgisi
print(matris.ndim)     # boyut sayısı
print(matris.size)     # toplam eleman sayısı
print("-----")

arr = np.zeros((3,3))   # sıfırlardan oluşan matris
print(arr)
print("-----")
arr2 = np.ones((2,4))   # birlerden oluşan matris
print(arr2)
print("-----")
arr3 = np.arange(0,10,2) # belirli aralıklar ile matris
print(arr3)
print("-----")
arr4 = np.linspace(0,1,5) # eşit aralıklı matris
print(arr4)
print("-----")
```

```
(2, 3)
2
6
-----
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
-----
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
-----
[0 2 4 6 8]
-----
[0.    0.25 0.5   0.75 1.   ]
-----
```

Python - Kütüphaneler

Numpy

```
# Matematiksel işlemler
a = np.array([1,2,3])
b = np.array([4,5,6])
```

```
print(a + b)
print(a * 2)
print(a ** 2)
print("-----")
```

```
# indexing
matris = np.array([
    [1,2,3],
    [4,5,6]
])
```

```
print(matris[0,1])
print("-----")
```

```
[5 7 9]
[2 4 6]
[1 4 9]
```

```
-----
2
-----
```

Python - Kütüphaneler

Numpy

```
a = np.array([1,2,3,4,5])

print(np.mean(a))    # ortalama
print(np.sum(a))     # toplam
print(np.max(a))     # maksimum
print(np.min(a))     # minimum
print("-----")

# Boyut değiştirme
a = np.arange(12)
print(a)
b = a.reshape(3,4)
print(b)
print("-----")
```

3.0
15
5
1

[0 1 2 3 4 5 6 7 8 9 10 11]
[[0 1 2 3]
 [4 5 6 7]
 [8 9 10 11]]

Python - Kütüphaneler

Numpy

```
# Rasgele sayı üretme
rasgeleDizi = np.random.rand(3,3)           # 0-1 arası
print(rasgeleDizi)
print("-----")

rasgeleDizi2 = np.random.randint(0,10,(3,3)) # 0-10 arası tam sayı
print(rasgeleDizi2)
```

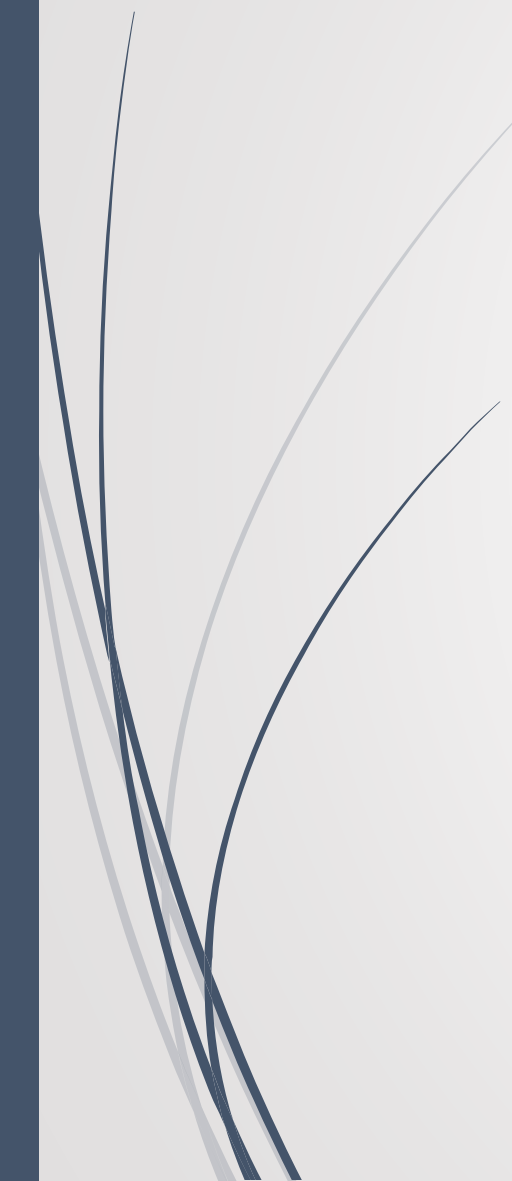
```
[[0.33723923 0.64526027 0.87102712]
 [0.76405017 0.93351726 0.65598427]
 [0.4835664  0.94277083 0.95234506]]
```

```
-----
[[1 2 9]
 [9 7 6]
 [2 0 9]]
```



Python - Kütüphaneler

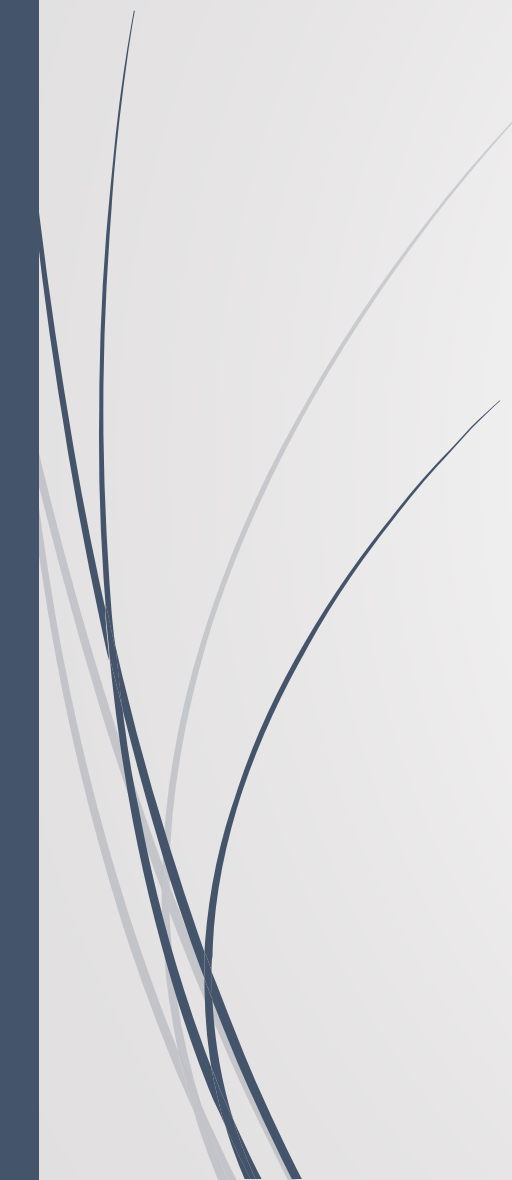
Pandas





Python - Kütüphaneler

Matplotlib





Python - Kütüphaneler

OS

